

A Federated Learning Approach for Speech Anti-Spoofing and Deepfake Detection

Abstract—Advances in speech synthesis and manipulation technologies have created significant challenges in maintaining the integrity and reliability of speech data, necessitating robust solutions for both anti-spoofing in Automatic Speaker Verification (ASV) systems and deepfake detection aimed at mitigating the misuse of synthetic speech for malicious practices. Federated learning, which allows for decentralized training across distributed data, offers a viable approach for improving privacy and adaptability in countermeasures. This study presents a federated learning technique for voice spoofing and deepfake detection, employing a wav2vec 2.0 frontend for feature extraction and a lightweight Multi-Scale Dilation Attention Network (MDAN) backend for multi-scale feature processing and classification. The model was trained on the ASVspoof 2019 Logical Access (LA) and Fake-or-Real (FoR) datasets. The proposed method achieved an equal error rate (EER) of 2.92% and an ASV-centric tandem detection cost function (t-DCF) score of 0.2672 on LA partition, and 2.99% EER on Deepfake (DF) partition of ASVspoof 2021 database, demonstrating its effectiveness.

I. INTRODUCTION

THE rapid evolution of audio synthesis and manipulation technology has presented significant challenges in preserving the integrity and authenticity of audio content (1). Such technologies, ranging from synthesized speech in automated speaker verification (ASV) systems (2) to sophisticated audio deepfakes, are a growing threat in voice automations and a variety of domains. In ASV systems, for example, spoofing attacks can exploit vulnerabilities to gain unauthorized access, with far-reaching implications for applications in finance, authentication, and personal security. Beyond ASV, audio deepfakes are rapidly being used to create realistic manipulations in social media, political discourse, and entertainment, spreading disinformation and raising ethical and security issues. Thus, there is a pressing need for effective countermeasures that can accurately detect these forged audio signals and protect against potential abuse. Traditional detection methods have become better at detecting speech alterations; however, the dynamic and adaptive nature of spoofing and deepfake generation approaches calls for flexible and scalable solutions. This is where federated learning comes into play. In contrast to centralized learning, which pools the data into a single repository for model training, federated learning allows the global model to be trained across multiple distributed data sources without transferring any actual data (3). In a speech anti-spoofing context, federated learning enables the model to be continuously updated with new audio patterns and variations encountered by individual devices, resulting in a more flexible and generalized detection system. A federated learning framework is beneficial in cases involving sensitive user data, such as voice biometrics and personally tailored voice assistants, where data privacy is of top concern. Federated learning protects user data by

locally training the model and sending updates of the model rather than raw data to a global model. This not only makes the model secure but also guards against attacks and other manipulations by preserving sensitive audio information locally. This work introduces a federated learning-based system for speech spoofing and deepfake detection. The system employs the feature-rich wav2vec 2.0 model (4) as a frontend and a novel Multi-Scale Dilation Attention Network (MDAN) at the backend to effectively learn multi-scale features. The major contributions of this work are as follows:

- 1) Development of an end-to-end federated learning architecture for speech spoofing and deepfake detection, improving privacy and scalability in ASV systems and beyond.
- 2) Introduction of the multi-scale dilation attention network (MDAN) back-end, which uses dilated convolutions and attention mechanisms to capture multi-scale audio features.
- 3) Instead of traditional flattening, global average pooling (GAP) is used after each dilation block to make the model lightweight while retaining spatial information and improving feature abstraction before classification.

Section 2 reviews the existing literature on the topic. In Section 3, the proposed methodology is outlined. Section 4 discusses the dataset used for training and evaluation purposes, and experimental settings. Section 5 presents the results obtained. Lastly, Section 6 concludes the study.

II. RELATED WORK

Numerous strategies have been developed over recent years for anti-spoofing in ASV systems and deepfake detection. For example, Xu Li et al. (5) present a unique channel-wise gated Res2Net (CG-Res2Net) architecture that aims to improve generalizability in detecting synthetic speech attacks. Traditional Res2Net systems, while effective, lack the flexibility to prioritize channel-specific information that may have key spoofing cues. The CG-Res2Net addresses this by introducing a channel-wise gating mechanism that dynamically selects relevant channels, improving detection across a wide range of spoofing attacks. Another study (6) introduces AASIST (Audio Anti-Spoofing Using Integrated Spectro-Temporal Graph Attention Networks). The method combines spectro-temporal graph attention layers into a single model to successfully capture spoofing artifacts across both spectral and temporal domains. A heterogeneous stacking graph attention layer (HSGAL) is introduced, along with a max graph operation (MGO) and an improved readout mechanism, which together enable AASIST to focus on key artifacts along varying spoofing methods. Hemlata Tak et al. (7) explore the effectiveness of using self-supervised learning (SSL) with a Wav2vec 2.0 frontend for spoofing detection. Though SSL frontends pre-trained on external data, such as wav2vec 2.0, were previously

restricted in ASVspoof challenges to ensure fair competition, recent research shows that they offer substantial improvements in generalization and robustness for countermeasure systems. When combined with the AASIST backend and enhanced by RawBoost data augmentation, the SSL frontend showed a remarkable reduction in error rates. A comparison study by Wang et al. (8) found that fine-tuning SSL models is critical for detecting spoofing across various front-end and back-end architectures. The study by Bo Wang et al. (9) proposed the FTDKD (Frequency-Time Domain Knowledge Distillation) framework, which involves training a high-quality audio "teacher" model alongside a compressed audio "student" model, allowing the student to inherit nuanced information lost during compression. Frequency-domain distillation gives priority to the retention of high-frequency features, while time-domain distillation uses sliced Wasserstein distance and contrastive loss to improve temporal details. Another paper (10) introduces a Temporal Channel Modeling (TCM) module to improve the XLSR-Conformer architecture for synthetic speech detection. The TCM module improves upon the standard Multi-Head Self-Attention (MHSA) mechanism by utilizing temporal-channel dependencies through innovative head token generation and classification token enrichment.

Federated learning has become an important standard for decentralized machine learning, allowing collaborative model training on scattered devices with guaranteed data privacy. Peiying Zhang et al. (11) address the issue of processing heterogeneous data that are sensitive to privacy produced by industrial Internet of Things (IIoT) devices where traditional centralized methods are inefficient and invasive to privacy. In an attempt to avoid these, the authors propose an advanced framework that combines Federated Learning (FL) and Deep Reinforcement Learning (DRL) in the form of the Deep Deterministic Policy Gradient (DDPG) algorithm. FL is employed to protect data privacy through decentralized model learning on local devices, whereas DRL optimizes the selection of high-quality IIoT nodes for model fusion to reduce communication overhead and enhance training efficiency. The framework is composed of three phases: initialization, local training, and global aggregation, thus addressing the heterogeneity of real-world IIoT environments. Another study (12) presents an extensive survey on the use of Federated Learning (FL) to mitigate cybersecurity issues, with a special focus on preserving privacy and decentralized data management. It discusses FL applications in detail across authentication, privacy, trust management, and attack detection, with the help of real-world use cases such as financial risk management, edge-based object detection, and anti-money laundering.

III. PROPOSED METHODOLOGY

The proposed methodology involves the following steps:

A. Data Augmentation

The RawBoost data augmentation (13) method is used. It is intended to mimic real-world audio distortions that frequently affect recordings in telephony and online contexts. Three types

of noise are added to the data, each of which addresses a specific form of variability encountered in audio transmission:

- 1) *Linear and Non-Linear (LnL) Convolutional Noise*: This augmentation simulates the channel distortions caused by telecommunication networks, such as encoding and compression artifacts. Using multi-band filters, RawBoost generates convolutional noise by applying randomly generated notch filters with changing center frequencies and bandwidths.
- 2) *Impulsive Signal-Dependent (ISD) Additive Noise*: This sort of noise indicates occasional spikes or irregularities in audio caused by the recording equipment issues, such as clipping or device malfunctions. The noise is signal dependent because the magnitude and even the sign of the artificial impulse depend on the current sample. In RawBoost, impulsive noise is introduced as random, instantaneous amplitude changes at specific points in the waveform.
- 3) *Stationary Signal-Independent (SSI) Additive Noise*: This form of noise, which is commonly caused by environmental factors like electromagnetic interference, is added uniformly across the signal. A colored noise is created by filtering white noise with a similar multi-band filter that simulates interference. Unlike impulsive noise, this stationary noise remains constant throughout the signal, resembling the steady, low-level noise seen in many recordings, especially in settings where ambient noise or poor-quality transmission lines are present.

The exact noise and filter parameters utilized are the same from the study by Hemlata Tak et al. (7).

B. Frontend

The Wav2vec 2.0 model (4) serves as the frontend for this anti-spoofing framework, playing a key role in extracting robust and informative features from raw audio input. It was developed for speech representation learning by Facebook AI. Unlike standard supervised models, which rely on labeled data, Wav2vec 2.0 learns representations directly from unlabeled audio using a contrastive learning approach. This self-supervised pretraining involves masking sections of the input audio representation and training the model to predict the masked portions based on context. By pretraining on vast volumes of raw audio, the model creates powerful feature representations that capture complex patterns in speech, such as phonetic, prosodic, and speaker-specific information. The architecture contains a convolutional feature encoder that transforms raw audio waveforms into latent feature representations, followed by a Transformer network that processes the latent representations using multi-head self-attention to capture contextual information. The encoder initially records low-level audio features, which are then refined by the Transformer layers further into higher-level abstractions. To provide a uniform input length, each signal is scaled to a fixed 64,600 timesteps (approximately 4 seconds); longer signals are truncated, while shorter signals are repeated until reaching the desired length, then trimmed as needed. When this 1-D signal passes through the wav2vec 2.0 convolutional feature encoder, the time axis is subsampled,

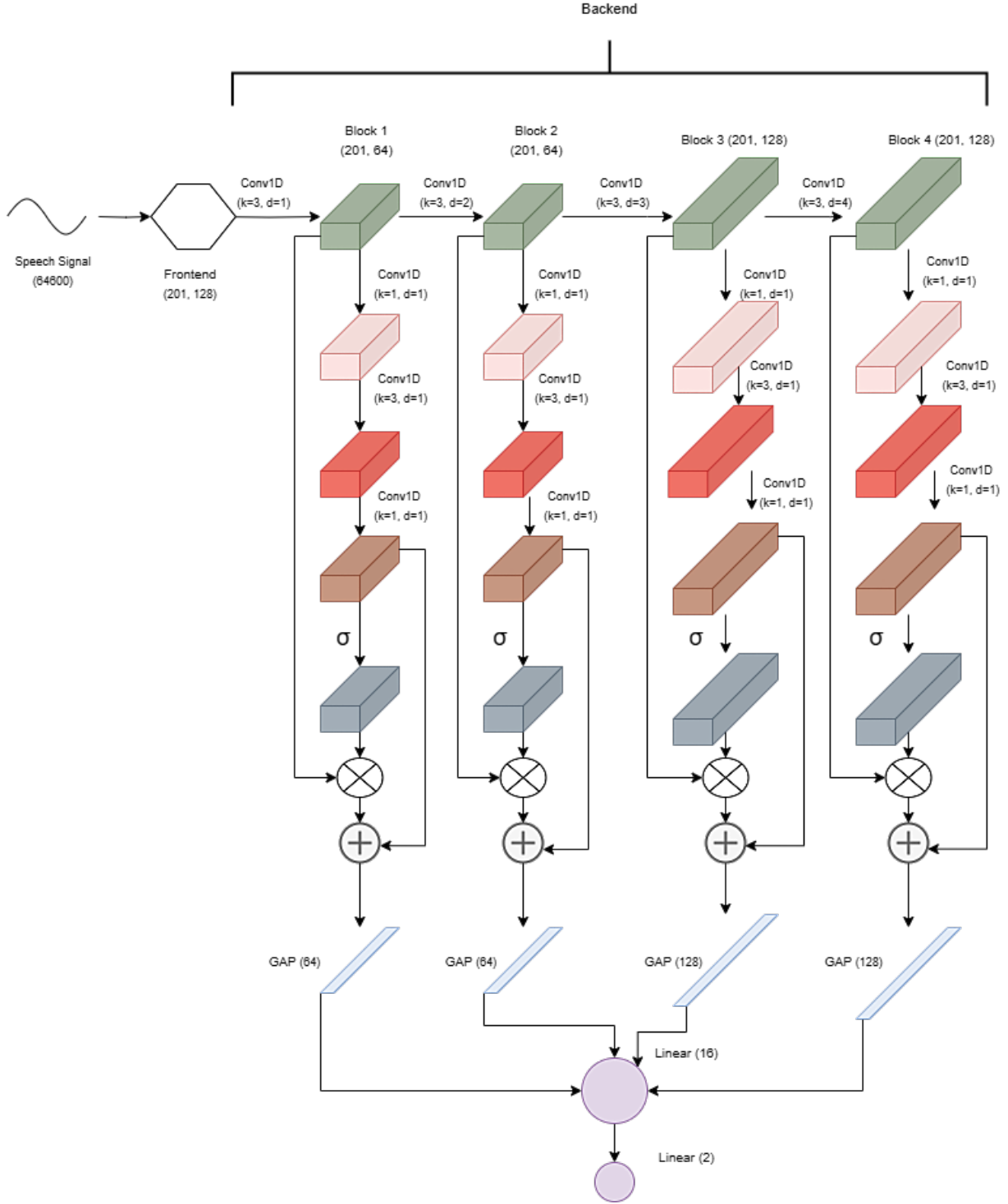


Fig. 1: Diagram illustrating proposed backend; where (x, y) defines shape of the block, k is kernel size and d the dilation rate.

yielding 201 feature frames. The encoder therefore produces a tensor of shape (201, 1024): 201 sequential frames, each represented by a 1024-dimensional feature vector. A linear transformation layer reduces the feature dimension from 1024 to 128, while maintaining the temporal length of 201. The frontend thus produces a feature tensor $X \in \mathbb{R}^{B \times C \times T}$, where $C = 128$ is the number of feature channels after dimensionality reduction, $T = 201$ represents the temporal dimension, and batch size B is taken as 16. This tensor serves as the input to the backend. The XLSR-300M Wav2Vec2 model has around 300 million parameters, and the linear transformation adds 131K parameters. The frontend is fine-tuned end-to-end during training.

C. Backend

The model's backend is designed to classify incoming input audio samples as either genuine or spoofed by processing the feature representations acquired from the wav2vec 2.0 frontend. It uses a Multi-Scale Dilation Attention Network (MDAN), which is made up of multiple convolutional attention blocks and a final classification step to extract informative representations that capture various receptive fields and then aggregate them for classification. The backend architecture is illustrated in figure 1. Each block takes a feature map X_{in} as input and returns an output X_{out} of different channel dimensionalities. The number of output channels is 64 for the initial two blocks, followed by 128 for the last two. For each block, a dilated convolution is applied to incorporate a greater temporal context without increasing the kernel size (14).

$$Z = f_{conv}(X_{in}) \quad (1)$$

Here f_{conv} is a 1D convolution with a specified dilation rate and appropriate padding to maintain temporal resolution. Mathematically, a dilated 1D convolution can be defined as follows:

$$y[n] = \sum_{i=0}^{k-1} x[n + i \cdot d] \cdot w[i] \quad (2)$$

Where $y[n]$ is the output signal at position n , $x[n]$ is the input signal, $w[i]$ represents the convolutional kernel weights, with $i = 0, 1, \dots, k-1$. k is the size of the convolutional kernel, and d is the dilation rate, determining the spacing between the kernel elements. It allows the network to capture broader contextual information without a proportional expansion in the number of parameters or computational complexity. The four blocks use dilation rates, $d = 1, 2, 3$, and 4 , respectively. A non-linear activation (LeakyReLU) is then applied:

$$\mathbf{Z}_\alpha = \text{LeakyReLU}(\mathbf{Z}) = \begin{cases} \mathbf{Z}, & \text{if } \mathbf{Z} \geq 0 \\ \alpha \cdot \mathbf{Z}, & \text{if } \mathbf{Z} < 0 \end{cases} \quad (3)$$

α is taken as 0.02. The primary goal of using Leaky ReLU is to mitigate the "dying ReLU" problem by permitting a modest, non-zero gradient while the unit is not active (i.e., when the input is negative). This facilitates better gradient flow during the training process. To refine the produced features, it $\mathbf{Z}_\alpha$ is

passed through a series of convolution layers. This sequence can be generalized as:

$$\mathbf{X} = f^1(f^3(f^1(\mathbf{Z}_\alpha))) \quad (4)$$

where f^1 and f^3 represent 1D convolutions with kernel sizes k of 1 and 3, respectively. These operations do not change the dimension. Then an attention mechanism is introduced to emphasize the most relevant parts of the input features. The attention weights are derived from the transformed feature map $\mathbf{X}$ using a sigmoid activation:

$$\mathbf{A} = \sigma(\mathbf{X}) \quad (5)$$

where σ is the element-wise sigmoid function. The attention weights $\mathbf{A}$ range between 0 and 1, indicating how much to highlight each feature at each timestep. This attention map is then applied to the original pre-transformation features $\mathbf{Z}_\alpha$ (the initial dilated convolution output) as follows:

$$\mathbf{X}_{att} = \mathbf{A} \odot \mathbf{Z}_\alpha \quad (6)$$

where $\odot$ denotes element-wise multiplication. Then the attention-weighted features $\mathbf{X}_{att}$ are combined with the transformed features $\mathbf{X}$ to form a residual-like structure:

$$\mathbf{X}_{out} = \mathbf{X} + \mathbf{X}_{att} \quad (7)$$

This ensures that the block's final output incorporates both the original learned representations and the attention-focused features, enriching the representation while preserving potentially useful information. After receiving the output of each block, a Global Average Pooling (GAP) operation (15) is applied along the temporal dimension:

$$\mathbf{h} = \text{GAP}(\mathbf{X}_{out}) \in \mathbb{R}^{B \times C_{out}} \quad (8)$$

where C_{out} is the number of output channels of that block. Global average pooling aggregates information across time, providing a compact, per-channel summary of the block's output features. Without the gating attention mechanism, GAP treats all spatial regions equally and simply averages them out. If there are irrelevant or noisy sections of the feature map, they can dilute the overall representation. Gated attention can lessen the impact of these noisy regions by lowering their values before the pooling operation, hence improving the quality of the pooled representation. When the truly important features are emphasized, the global average now becomes a more discriminative summary. This can result in better downstream classification performance. The process described above is repeated for multiple blocks, each with different dilation rates and number of output channels. Suppose the produced pooled outputs are $\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3$, and $\mathbf{h}_4$. They are concatenated along the channel dimension as:

$$\mathbf{H} = [\mathbf{h}_1; \mathbf{h}_2; \mathbf{h}_3; \mathbf{h}_4] \quad (9)$$

This provides a combined feature vector $\mathbf{H} \in \mathbb{R}^{B \times D}$, where D is the sum of the dimensions of all block outputs. Here, $D = 64 + 64 + 128 + 128 = 384$. A fully connected layer is then applied to reduce the dimensionality and capture interactions among the concatenated features:

$$\mathbf{u} = f_{dense}(\mathbf{H}) \quad (10)$$

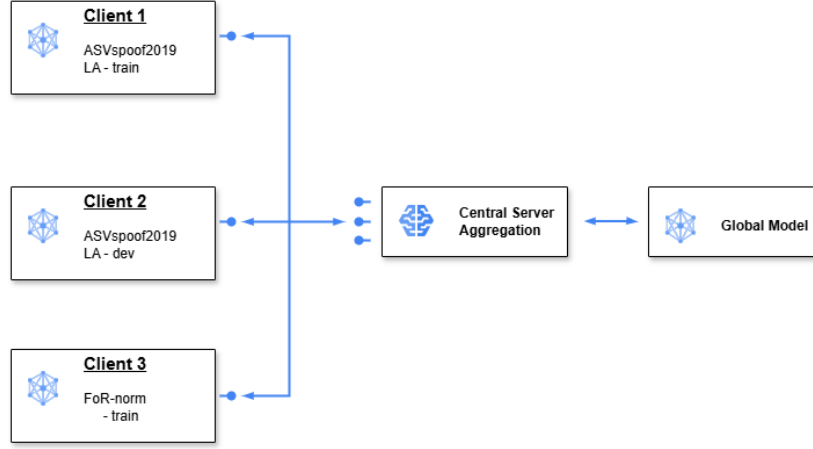


Fig. 2: Diagram illustrating federated learning.

where f_{dense} is a linear transform followed by the non-linear activation LeakyReLU. Finally, a linear output layer is used to produce class probabilities via a softmax function:

$$\mathbf{y} = \text{softmax}(\mathbf{W}_o \mathbf{u} + \mathbf{b}_o) \quad (11)$$

with $\mathbf{W}_o \in R^{num_classes * dim(u)}$ and $\mathbf{b}_o \in R^{num_classes}$. Here, the dimension of $\mathbf{u}$ is taken as 16. The total number of parameters in the backend model is about 323K.

D. Federated Learning

In this study, a federated learning technique, as shown in figure 2, is used to train the proposed anti-spoofing and deep-fake detection model. The federated setup has three clients, each aligned with a distinct data partition: the ASVspoof 2019 LA train subset, the ASVspoof 2019 LA development subset, and the training division of the Fake-or-Real normalized (FoR-norm) corpus (16). Each client performs local model updates on its respective audio data, while the central server aggregates their contributions to build a global model without exchanging the raw data. The process starts with initializing a global model consisting of the wav2vec 2.0 frontend for feature extraction and the Multi-Scale Dilation Attention Network (MDAN) backend for classification. The central server distributes the initial global model weights to each client, who then trains the model locally on their respective datasets and sends back updated weights after training. The weights are then aggregated and sent to client models again for training. This process continues for a set number of federated learning rounds. Exact training details is given in Section 4C.

IV. EXPERIMENTAL SETUPS

A. Train Datasets

1) *ASVspoof 2019 (2) Logical Access (LA) – Train and Development Partitions*: The ASVspoof 2019 LA dataset is derived from the VCTK base corpus (17) and is sampled at 16 kHz with 16-bit resolution. It supplies 25,380 training utterances and 24,844 development utterances, covering a broad spectrum of text-to-speech (TTS) and voice-conversion (VC) spoofing attacks, utilizing 17 unique systems—such

TABLE I: Number of speech samples per class in the training datasets.

Dataset	Genuine samples	Forged samples
ASVspoof 2019 LA-train	2 580	22 800
ASVspoof 2019 LA-dev	2 548	22 296
FoR norm - train	26 900	26 900

as waveform concatenation, neural waveform modeling (e.g., WaveNet (18)), and complex sequence-to-sequence neural networks. The attacks are categorized into six known and eleven unknown spoofing classes. The known attacks consist of 2 VC systems and 4 TTS systems, and the unknown attacks consist of 2 VC, 6 TTS, and 3 hybrid TTS-VC systems injecting synthetic speech into VC systems.

2) *Fake-or-Real dataset - Train Division (16)*: It is a large-scale collection of natural and synthetic speech, featuring both human recordings and advanced text-to-speech (TTS) systems. It integrates data from multiple speech corpora, such as CMU Arctic, LJSpeech, and VoxForge, along with samples produced by cutting-edge synthesis techniques like Google Wavenet and Deep Voice 3. In this study, the for-norm version is used, which is specifically balanced for gender and class while also being normalized in terms of sampling rate, volume, and channels. This version ensures fair and uniform tests of models, reducing potential biases in the dataset. The training division used here represents 77.73% of the total for-norm corpus.

Class division details of the train dataset is given in Table 1.

B. Evaluation Datasets

1) *ASVspoof 2019 LA – Evaluation Partition*: The evaluation split supplies 71,237 utterances created with the same 17 spoofing systems described above, enabling assessment on data disjoint from the training procedure.

2) *ASVspoof 2021 LA and DeepFake (DF) (19) Datasets*: The Logical Access (LA) dataset addresses variations due

to encoding, compression, and transmission over voice-over-IP (VoIP) and public switched telephone networks (PSTN). The dataset is specifically designed to mimic real automatic speaker verification (ASV) conditions, where authentication frequently takes place over telecommunication networks. To facilitate this simulation, the dataset contains seven distinct encoding scenarios, including legacy telephony codecs such as G.722, a-law, and μ -law, as well as contemporary IP streaming codecs such as OPUS. In contrast, the DF dataset is specifically designed to address the increasing threats of manipulated speech and AI-generated voice deepfakes in media, social media, and online content. Spoofed audio is created using over 100 distinct voice conversion (VC) and text-to-speech (TTS) algorithms, including both legacy vocoder-based synthesis techniques and more recent deep learning-based voice cloning methods. One of the primary challenges inherent in this dataset concerns the implications of compression and transcoding, mimicking how deepfakes are frequently encountered in real-world applications. This dataset contains nine distinct compression scenarios, including formats such as MP3, M4A (AAC), and OGG Vorbis, each with different bitrate configurations. Moreover, some files are double compressed, where audio is transcoded between formats, mimicking conditions frequently encountered on online platforms where content is uploaded, re-encoded, and redistributed.

3) *FoR norm – Validation and Testing Divisions*: The remaining 22.27% of the for-norm corpus, comprising the validation and testing partitions, is reserved for evaluation to gauge the model’s out-of-sample performance.

4) *In-the-Wild dataset (20)*: This collection aggregates approximately 38 hours of authentic and synthetic speech scraped from public sources such as social media and video-sharing platforms. It features 58 speakers (politicians, celebrities, etc.), each with roughly 23 min of bona-fide and 18 min of spoofed audio, providing a challenging cross-database generalizability test.

C. Experimental Settings and Metrics

During the first federated training round, each client conducts three local epochs on its designated data subset—the ASVspoof 2019 LA train partition, the ASVspoof 2019 LA development partition, or the FoR-norm training split. The training is carried out using the Adam optimizer with a learning rate of $1e-5$, $1e-4$ weight decay and employs Stationary Signal-Independent (SSI) additive noise for data augmentation. Each client minimises a cross-entropy objective, a 1 : 9 spoof-to-bonafide weighting is applied for the ASVspoof 2019 LA clients to offset class imbalance (21), while the FoR norm client uses the standard unweighted form owing to its balanced label distribution. . The weighted cross-entropy loss is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [w_1 \cdot y_i \cdot \log(p_i) + w_0 \cdot (1 - y_i) \cdot \log(1 - p_i)] \quad (12)$$

Where labels $y_i \in \{0, 1\}$, p_i is the predicted probability for the spoofed sample class ($y_i = 1$) and its weight $w_1 = 0.1$. $w_0 = 0.9$ for the bonafide class ($y_i = 0$) with predicted

probability $1 - p_i$. At the completion of the first training round, each client produces a set of updated model weights. These weights are then delivered to the central server, where they are aggregated using a simple averaging mechanism before sending back to all the clients for the second round of training. In the second round, each client trains the model for an additional 4 epochs using the revised global model weights as initialization and with a lowered learning rate of $1e-6$. Linear and non-linear (LnL) convolutive noise, along with impulsive signal-dependent (ISD) additive noise, is used for this round of training as data augmentation. Each client again produces another set of updated model weights. These weights are sent to the central server and aggregated in the same way as in the first round. A third round of training is done for 3 epochs, at the same learning rate while applying only stationary signal-independent (SSI) additive noise. The weights aggregated after this stage form the final global model. The evaluation focuses on two widely known performance metrics: equal error rate (EER)% and the min tandem detection cost function (t-DCF) (22). The Equal Error Rate (EER) is a standard and widely accepted metric for evaluating classification systems, particularly in anti-spoofing and verification tasks. It represents the point where the false acceptance rate (FAR) equals the false rejection rate (FRR). EER can be mathematically defined as:

$$FRR(\theta) = \frac{|\{s \in S_{\text{target}} \mid s < \theta\}|}{|S_{\text{target}}|} \quad (13)$$

$$FAR(\theta) = \frac{|\{s \in S_{\text{nontarget}} \mid s \geq \theta\}|}{|S_{\text{nontarget}}|} \quad (14)$$

$$\theta^* = \arg \min_{\theta} |FAR(\theta) - FRR(\theta)| \quad (15)$$

$$EER = FAR(\theta^*) = FRR(\theta^*) \quad (16)$$

where θ is the decision threshold, S_{target} is the set of genuine (positive) scores, $S_{\text{nontarget}}$ is the set of impostor (negative) scores, FRR is the false rejection rate, FAR is the false acceptance rate, and $|\cdot|$ denotes the cardinality of a set.

A lower EER suggests a more precise system, as it implies fewer errors in both accepting spoofed samples and rejecting genuine samples. EER returns a single scalar value that is simpler to interpret and compare across models. For the ASVspoof 2021 LA task, in addition to EER, the tandem detection cost function (t-DCF) is used. This statistic is designed to assess the performance of spoofing detection systems within the context of an integrated ASV pipeline. The t-DCF takes into consideration the operational cost of both the spoofing detection system and the ASV system, making it a more practical evaluation for real-world applications. Lower t-DCF scores indicate better alignment with the ASV system’s performance.

V. RESULTS AND DISCUSSION

Configuration-level comparisons are conducted on the ASVspoof 2021 dataset, covering both the Logical Access

(LA) anti-spoofing and DeepFake (DF) tracks, while comparative results against prior work are reported on the test sets those studies employed. LA and DF tracks are distinct but complementary, allowing for a comprehensive analysis of the model’s effectiveness in both anti-spoofing and deepfake detection tasks. While both performance metrics, EER and t-DCF, are used for the LA task, only EER is reported for the DF task, as there is no associated ASV system in this case.

TABLE II: Proposed-method results on the ASVspoof 2021 evaluation sets with different noise configurations during federated-learning rounds.

Data-augmentation noise	EER% (LA)	min t-DCF (LA)	EER% (DF)
No data augmentation	7.16	0.3820	9.14
SSI	5.83	0.3407	2.55
LnL + ISD	0.98	0.2047	5.91
SSI ; LnL + ISD ; SSI	2.92	0.2672	2.99

The model is tested under different noise conditions. Several configurations are tested: no noise, only SSI additive noise, LnL convolutive noise combined with ISD additive noise, and swapping of augmentations between federated rounds. Table 2 presents the results under the different noise configurations. It is evident that all cases of noise augmentation yielded better results compared to the baseline model trained without any noise augmentation. Among the tested configurations, SSI additive noise resulted in the lowest EER for the DF task, indicating its effectiveness in detecting synthetic speech. However, SSI noise performed poorly for the LA task, with results closer to the no-noise baseline. On the other hand, LnL convolutive noise paired with ISD additive noise produced the best results for the LA task, achieving significantly lower EER and t-DCF scores compared to SSI additive noise. However, this configuration did not perform as well for the DF task relative to other cases. The best overall performance was obtained by strategically combining these noise types in the federated learning framework, with SSI noise used in the first training round, LnL + ISD noise in the second round, and SSI noise again in the third round.

TABLE III: MDAN-backend results with different frontend structures.

Frontend	EER% (LA)	min t-DCF (LA)	EER% (DF)
Sinc-layer	7.96	0.3977	21.09
Mel spectrogram	11.68	0.4905	22.75
Wav2vec 2 (Proposed)	2.92	0.2672	2.99

The study also compares the performance of the proposed model, which utilizes a wav2vec 2.0 frontend and a Multi-Scale Dilation Attention Network (MDAN) backend, against alternative front-end and back-end configurations. These comparisons provide insight into the effectiveness of each component in the pipeline. Table 3 compares the performance of the proposed wav2vec 2.0 frontend with two other frontend architectures: a sinc-layer-based frontend and a Mel spectrogram-

TABLE IV: Wav2vec 2.0 performance with different backend architectures.

Backend	EER% (LA)	min t-DCF (LA)	EER% (DF)
AAIST	2.79	0.2663	4.30
Proposed, standard conv (all 1-D convolutions, $d=1$)	3.41	0.2791	6.48
Proposed, no attention (dilated convs, no attention)	3.07	0.2706	5.12
MDAN (proposed) (dilated convs + attention)	2.92	0.2672	2.99

based frontend, both paired with the MDAN backend. The results demonstrate that the wav2vec 2.0 frontend consistently outperformed the alternatives across both the LA and DF tasks. Its self-supervised representation learning capability captures richer and more robust features, resulting in better classification performance. The effectiveness of the proposed MDAN backend was tested against three other backends, as shown in Table 4. The first alternative, AAIST (6), employs integrated spectro-temporal graph attention networks. Two in-house ablations look at how MDAN’s key design choices affect the results: Standard-conv keeps the overall topology but replaces every dilated convolution with a regular 1-D kernel (dilation = 1). No-attention keeps the multi-scale dilated stacks but takes out the gated attention blocks. While the AAIST backend produced a marginally better result for the LA task, achieving an EER of 2.79% compared to MDAN’s 2.92%, the proposed MDAN backend demonstrated superior performance for the DF task. Specifically, MDAN achieved an EER of 2.99% for DF detection compared to AAIST’s 4.30%. Dilated convolutions enlarge the receptive field without additional parameters, capturing longer-range temporal cues that standard kernels miss. The inclusion of attention mechanisms in MDAN plays a crucial role in capturing subtle synthetic speech patterns, particularly in challenging scenarios like deepfake detection.

Table 5 compares the proposed method against results reported in other studies. The model delivers an EER as low as 0.02% on the ASVspoof2019 LA evaluation partition. On the more recent ASVspoof2021 tracks, performance is roughly comparable to the state of the art methods. The raw federated model achieves 2.92% on the LA track and 2.99% on the DF track, but when the data-augmentation noise is adjusted independently for each track, those numbers fall to 0.98% (LA) and 2.55% (DF) (values shown in parenthesis). Truong et al. (10) and Rosello et al. (23) also use a similar track-specific augmentation approach. To promote a model that generalizes across all evaluation sets without re-configuration, the federated pipeline uses a single augmentation schedule, which alternates LnL + ISD and SSI noises across rounds, as opposed to such fine-grained tuning. A centralized baseline that was trained using the same architecture across all three training corpora outperforms on the ASVspoof2021 tracks (2.43% LA, 2.86% DF), but falls short on the ASVspoof2019 LA, Fake-or-Real, and In-the-Wild tracks. Better cross-corpus

TABLE V: Comparison of the proposed method with other systems using the EER % metric. (For the proposed system, the value in parentheses is the EER % after data augmentation optimised separately for that ASVspoof 2021 track).

System	ASVspoof2019 LA	ASVspoof2021 LA	ASVspoof2021 DF	Fake-or-Real	In-the-Wild
Tak <i>et al.</i> (7)	—	0.82	6.64	—	—
Rosello <i>et al.</i> (23)	—	0.97	2.58	—	—
Truong <i>et al.</i> (10)	—	1.18	2.25	—	—
Zbezhkhovska (24)	—	—	29.67	35.30	34.55
Yang <i>et al.</i> (25)	0.11	0.94	1.40	—	4.91
Huang <i>et al.</i> (26)	0.20	1.87	3.38	3.69	5.99
Proposed (Federated training)	0.02	2.92 (0.98)	2.99 (2.55)	2.30	6.61
Centralized training	0.04	2.43	2.86	2.62	9.13

robustness, scalability, and privacy guarantees are features that centralised learning cannot provide, but the federated variant does. Although the proposed system does not post the absolute lowest EER on every dataset, it consistently maintains an EER below 3% on all but the cross-database In-the-Wild test, a modest drop considering the domain shift.

VI. CONCLUSION

The paper presented a federated learning framework for voice spoofing and deepfake detection, integrating the wav2vec 2.0 frontend for feature extraction with an optimized Multi-Scale Dilation Attention Network (MDAN) backend for multi-scale feature processing and classification. The federated learning architecture enables decentralized training while ensuring data privacy, thus making it extremely applicable to applications involving sensitive speech data. The critical role of data augmentation was demonstrated using experiments with various combinations of noise. The wav2vec 2.0 self-supervised learning frontend effectively learns robust speech representations, which complement the MDAN backend perfectly in order to improve detection efficacy. The gated attention mechanism integrated into MDAN, along with global average pooling, enhances feature representation further by suppressing the influence of irrelevant or noisy components of the feature map, leading to a more discriminative pooled summary. Future work could investigate more sophisticated self-supervised learning (SSL) frontends to further improve model performance, especially under more realistic real-world scenarios. Additionally, exploring new data augmentation techniques and integrating them into federated learning frameworks seamlessly could potentially improve generalization on varied datasets.

REFERENCES

- [1] K. T. Mai *et al.*, “Warning: Humans cannot reliably detect speech deepfakes,” *PLOS ONE*, vol. 18, no. 8, p. e0285333, 2023.
- [2] A. Nautsch *et al.*, “ASVspoof 2019: Spoofing countermeasures for the detection of synthesized, converted and replayed speech,” *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 3, no. 2, pp. 252–265, 2021.
- [3] P. Kairouz *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [4] A. Baevski *et al.*, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 12449–12460, 2020.
- [5] X. Li *et al.*, “Channel-wise gated res2net: Towards robust detection of synthetic speech attacks,” arXiv:2107.08803, 2021.
- [6] J.-w. Jung *et al.*, “Aasist: Audio anti-spoofing using integrated spectro-temporal graph attention networks,” in *ICASSP, IEEE*, 2022.
- [7] H. Tak *et al.*, “Automatic speaker verification spoofing and deepfake detection using wav2vec 2.0 and data augmentation,” arXiv:2202.12233, 2022.
- [8] X. Wang and J. Yamagishi, “Investigating self-supervised front ends for speech spoofing countermeasures,” arXiv:2111.07725, 2021.
- [9] B. Wang *et al.*, “FTDKD: Frequency-time domain knowledge distillation for low-quality compressed audio deepfake detection,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.
- [10] D.-T. Truong *et al.*, “Temporal-channel modeling in multi-head self-attention for synthetic speech detection,” arXiv:2406.17376, 2024.
- [11] P. Zhang, C. Wang, C. Jiang, and Z. Han, “Deep reinforcement learning assisted federated learning algorithm for data management of iiot,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 12, pp. 8475–8484, 2021.
- [12] M. Alazab, S. P. Rm, M. Parimala, P. K. R. Maddikunta, T. R. Gadekallu, and Q.-V. Pham, “Federated learning

- for cybersecurity: Concepts, challenges, and future directions,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 5, pp. 3501–3509, 2021.
- [13] H. Tak *et al.*, “Rawboost: A raw data boosting and augmentation method applied to automatic speaker verification anti-spoofing,” in *ICASSP*, IEEE, 2022.
 - [14] F. Yu, “Multi-scale context aggregation by dilated convolutions,” arXiv:1511.07122, 2015.
 - [15] M. Lin, “Network in network,” arXiv:1312.4400, 2013.
 - [16] R. Reimao and V. Tzerpos, “FoR: A dataset for synthetic speech detection,” in *SpeD*, IEEE, 2019.
 - [17] C. Veaux, J. Yamagishi, and K. MacDonald, “CSTR VCTK Corpus: English multi-speaker corpus for CSTR voice cloning toolkit,” tech. rep., University of Edinburgh, 2017. [sound].
 - [18] A. van den Oord *et al.*, “Wavenet: A generative model for raw audio,” arXiv:1609.03499, 2016.
 - [19] J. Yamagishi *et al.*, “ASVspoof 2021: Accelerating progress in spoofed and deepfake speech detection,” in *ASVspoof 2021 Workshop*, 2021.
 - [20] N. M. Müller *et al.*, “Does audio deepfake detection generalize?,” arXiv:2203.16263, 2022.
 - [21] T. H. Phan and K. Yamamoto, “Resolving class imbalance in object detection with weighted cross entropy losses,” arXiv:2006.01413, 2020.
 - [22] T. Kinnunen *et al.*, “Tandem assessment of spoofing countermeasures and automatic speaker verification: Fundamentals,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2195–2210, 2020.
 - [23] E. Rosello, A. G. Alanís, A. M. Gomez, A. M. Peinado, N. Harte, J. Carson-Berndsen, and G. Jones, “A conformer-based classifier for variable-length utterance processing in anti-spoofing,” in *Interspeech*, vol. 2023, pp. 5281–5285, 2023.
 - [24] U. Zbezhkhovska, “Deepfake audio detection using yolov8 with mel-spectrogram analysis: A cross-dataset evaluation,” *Radio Electronics, Computer Science, Control*, no. 1, pp. 153–163, 2025.
 - [25] M. Yang, Y. Gu, Q. He, Y. Li, P. Zhang, Y. Chen, Z. Wang, H. Zhu, J. Liu, and W. Wang, “Generalizable audio deepfake detection via hierarchical structure learning and feature whitening in poincaré sphere,” *arXiv preprint arXiv:2508.01897*, 2025.
 - [26] W. Huang, X. Liu, X. Wang, J. Yamagishi, and Y. Qian, “From sharpness to better generalization for speech deepfake detection,” *arXiv preprint arXiv:2506.11532*, 2025.